

PROJECT DOCUMENTATION

AWS COST OPTIMIZATION SYSTEM

Automated AWS Cost Monitoring, Analytics
and Optimization Platform

Created By
Om Doifode



PROJECT OVERVIEW

The AWS Cost Optimization System is a cloud-native project that automatically monitors your AWS billing data, sends email alerts when your spending goes above a set limit, stores daily cost reports inside an Amazon S3 bucket, and displays all of that information through a professional Streamlit web dashboard. The dashboard is hosted on an AWS EC2 server and runs continuously using PM2, which means it stays online even after you disconnect from the terminal or restart the server.

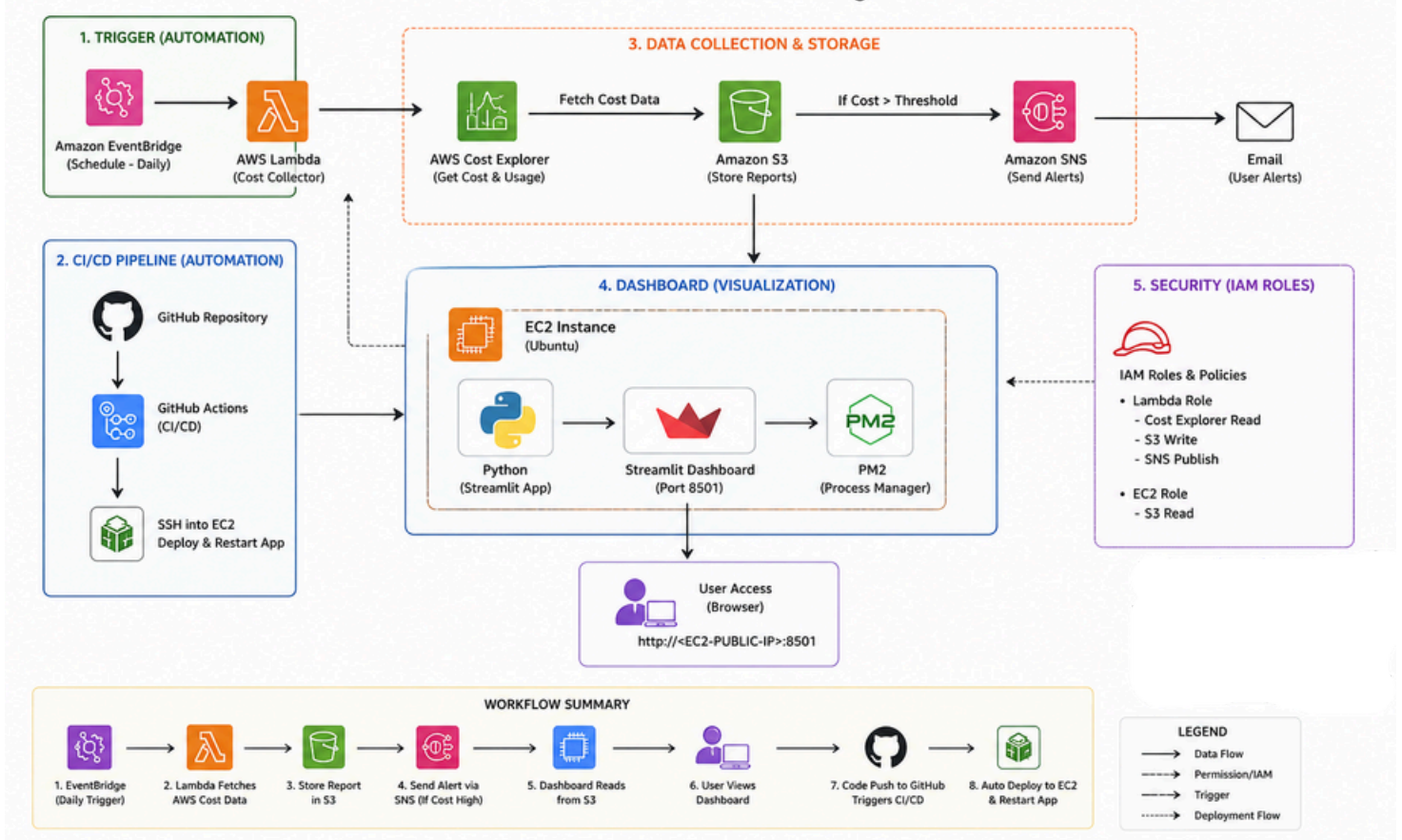
The project uses AWS Lambda for automatic billing data collection, Amazon EventBridge to schedule the Lambda function daily, Amazon SNS for email alerts, Amazon S3 to store the reports, and AWS Cost Explorer to pull the actual billing data from your AWS account. The entire codebase is version-controlled using Git and GitHub, and the deployment follows a real-world DevOps workflow including CI/CD with GitHub Actions.

SYSTEM ARCHITECTURE

The following diagram shows how all the components of this project are connected and how data flows from AWS billing to your browser:

AWS Cost Optimization System

Architecture & Workflow Diagram



STEP-BY-STEP IMPLEMENTATION

PHASE 1: Project Setup on Local Machine

Step 1: Create the Project Folder and File Structure

1. Open your Ubuntu terminal (Ctrl+Alt+T) and navigate to your home directory.
2. Create the main project folder named **aws-cost-optimization-system** along with all required subfolders and empty files.
3. This sets up a professional, industry-standard folder layout before writing any code.

```
cd ~
mkdir aws-cost-optimization-system
cd aws-cost-optimization-system
mkdir src docs tests architecture .github
mkdir .github/workflows
touch README.md requirements.txt dashboard.py
touch src/lambda_handler.py src/cost_analyzer.py src/notifier.py
touch .github/workflows/deploy.yml
```

4. Run **tree** command to verify the folder structure (install with **sudo apt install tree -y** if needed).

Step 2: Open Project in VS Code and Add File Content

1. Open VS Code directly from the terminal using **code .** command - this opens the entire project folder as a workspace.
2. Then open each file from the left sidebar and add its content.
3. This is where you write the initial code, configuration, and documentation.

Files to fill:

- **README.md** – Project overview, features list
 - **requirements.txt** – Add: boto3, streamlit, pandas, plotly
 - **src/cost_analyzer.py** – Cost analysis logic function
 - **src/notifier.py** – Notification print function
 - **dashboard.py** – Basic Streamlit UI setup
4. Save each file with **Ctrl+S**. Install dependencies by opening the VS Code terminal (**Terminal → New Terminal**) and running:

```
pip install -r requirements.txt
```

Step 3: Run the Streamlit Dashboard Locally

1. Test the dashboard locally before deploying to AWS. Run the following command in the VS Code terminal.
2. The dashboard will open automatically in your browser at **http://localhost:8501**. This confirms the project is set up correctly and the UI is working on your local machine.

```
streamlit run dashboard.py
```

3. Once verified, stop the server by pressing **Ctrl+C** in the terminal.

Step 4: Initialize Git and Push to GitHub

1. Initialize a local Git repository, connect it to a new GitHub repo, and push your code. Create the repository on GitHub as aws-cost-optimization-system (Public, no README, no gitignore). This is essential for version control, collaboration, and later deploying updated code to EC2 using git pull.

```
git init
git add .
git commit -m "Initial project setup"
git remote add origin https://github.com/Username/repo.git
git branch -M main
git push -u origin main
```

Note: When GitHub asks for a password during push, use a Personal Access Token (PAT) - not your GitHub password. Generate it at: GitHub → Settings → Developer Settings → Personal Access Tokens → Tokens (classic) → Enable repo scope.

PHASE 2 – AWS Infrastructure Setup

Step 5: Configure AWS CLI on Local Machine

1. Download and install the AWS CLI MSI Installer for Windows (64-bit) from the AWS website, then restart VS Code.
2. Once installed, configure the CLI with your AWS credentials. This connects your terminal to your AWS account so you can create and manage cloud resources from the command line.

```
aws configure
# AWS Access Key ID:
# AWS Secret Access Key:
# Default region name: e.g. ap-south-1
# Default output format: json
```

3. Verify the connection:

```
aws sts get-caller-identity
```

Note: To create Access Keys: AWS Console → IAM → Users → Security Credentials → Create Access Key → Select CLI. Never share your Access Key or Secret Key publicly

Step 6: Create SNS Topic and Subscribe Your Email

1. Amazon SNS (Simple Notification Service) is used to send email alerts whenever AWS cost goes above the threshold.
2. First, create a topic to act as the notification channel, then subscribe your email address to it. You will receive a confirmation email from AWS - you must click the confirmation link before any alerts can be delivered

```
aws sns create-topic --name aws-cost-alerts
# Copy the TopicArn from the output - you will need it in the Lambda
code

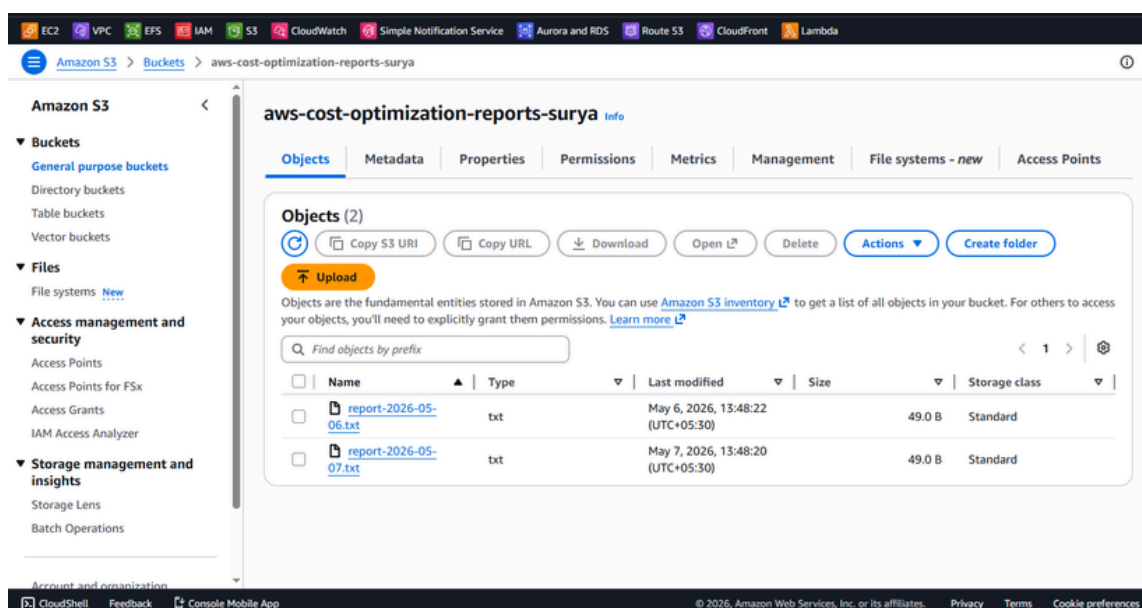
aws sns subscribe \
--topic-arn arn:aws:sns:ap-south-1:YOUR_ACCOUNT_ID:aws-cost-alerts \
--protocol email \
--notification-endpoint your@email.com
```

3. Open your email inbox and click the **Confirm Subscription** link in the confirmation email.

Step 7: Create S3 Bucket for Cost Reports

1. Create an S3 bucket to store the weekly cost report files generated by the Lambda function. Each time Lambda runs, it saves a text report file in this bucket. The Streamlit dashboard later reads these files from S3 to display the cost trend charts and metrics

```
aws s3 mb s3://aws-cost-optimization-reports-surya
```

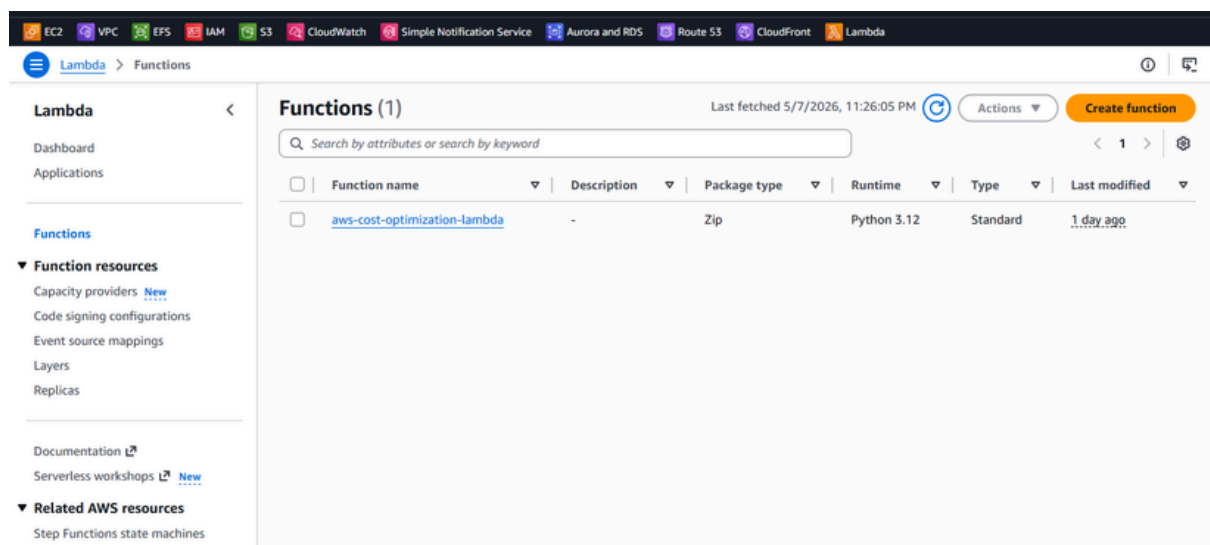


Note: S3 bucket names are globally unique across all AWS accounts. If the name already exists, add your name or a unique number at the end to avoid conflicts

Step 8: Create Lambda Function on AWS Console

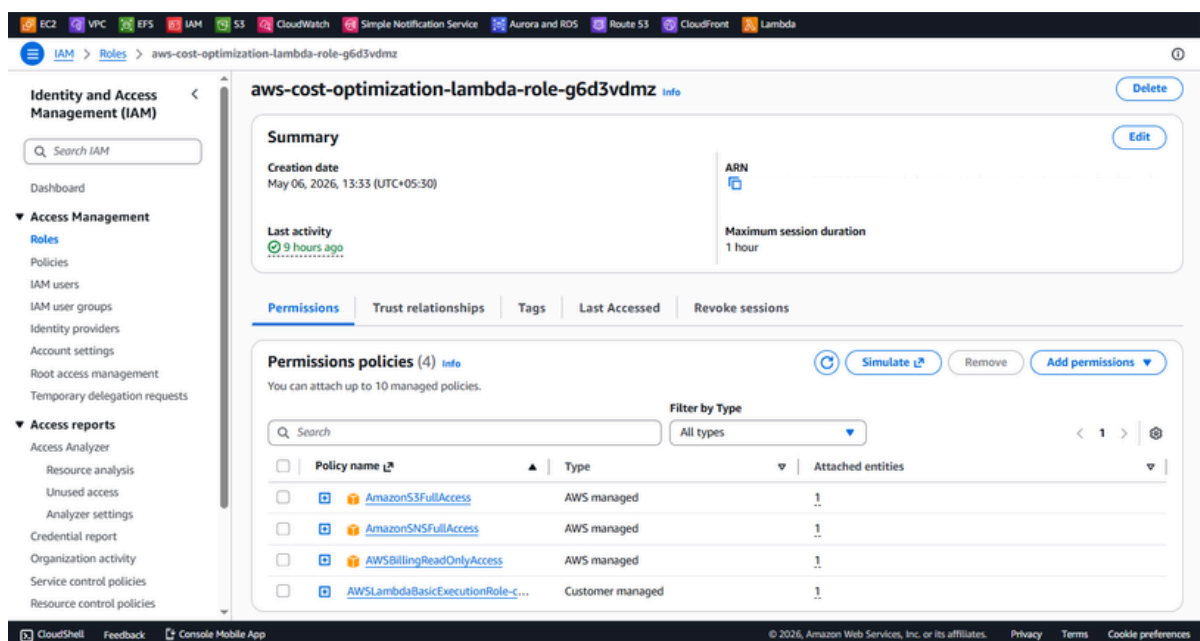
1. AWS Lambda is the core automation engine of this project. It runs Python code on a schedule without needing a dedicated server.
2. Go to **AWS Console → Lambda → Create Function** and configure it using the values below. Lambda will call the Cost Explorer API to fetch billing data, check if cost exceeds the threshold, send an SNS email alert, and save a report file in S3.

Field	Value
Function name	aws-cost-optimization-lambda
Runtime	Python 3.12
Execution role	Create a new role with basic Lambda permissions



Step 9: Attach Required IAM Permissions to Lambda Role

1. By default, the Lambda execution role only has basic logging permissions. You need to attach additional policies so Lambda can access Cost Explorer (billing data), SNS (to send alerts), and S3 (to save reports).
2. Go to: **Lambda → Configuration → Permissions → Click the role name → Add Permissions → Attach Policies**. and attach all three policies one by one.
 - **AmazonSNSFullAccess**
 - **AmazonS3FullAccess**
 - **AWSBillingReadOnlyAccess** (also try: **CostExplorerReadOnlyAccess** if not found)



Step 10: Add Lambda Function Code and Deploy

1. Go to **Lambda** → **Code**, delete the default code, and paste the complete Python code below. Replace the placeholder SNS ARN with your actual Topic ARN copied from Step 6, and confirm the S3 bucket name matches what you created in Step 7.

1. Click Deploy to save the function.

```
import json
import boto3
import datetime

sns = boto3.client('sns')
ce = boto3.client('ce')
s3 = boto3.client('s3')

TOPIC_ARN = "arn:aws:sns:ap-south-1:YOUR_ACCOUNT_ID:aws-cost-alerts"
BUCKET_NAME = "aws-cost-optimization-reports-surya"

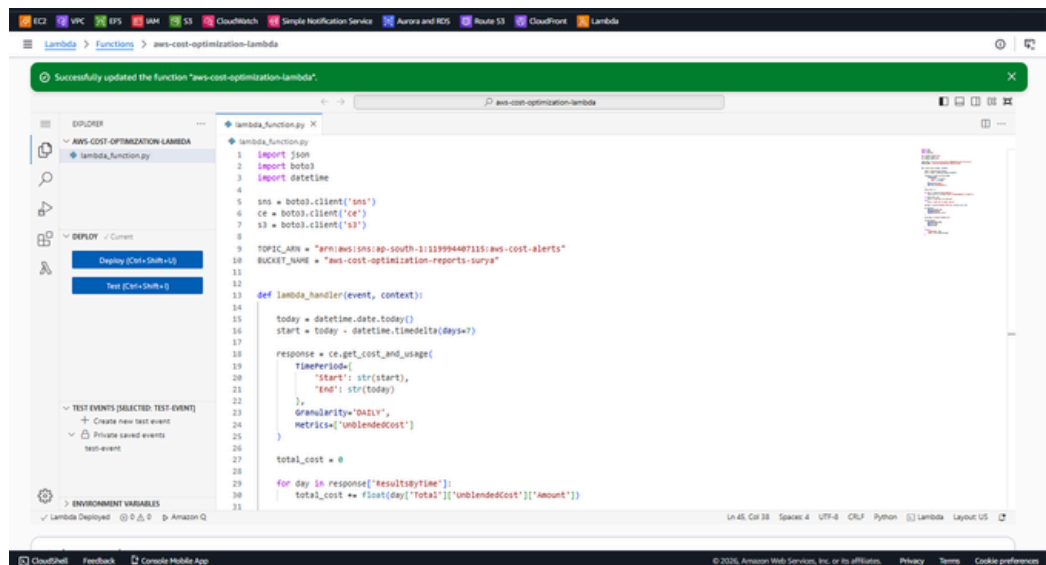
def lambda_handler(event, context):
    today = datetime.date.today()
    start = today - datetime.timedelta(days=7)
    response = ce.get_cost_and_usage(
        TimePeriod={'Start': str(start), 'End': str(today)},
        Granularity='DAILY', Metrics=['UnblendedCost'] )
    total_cost = 0
    for day in response['ResultsByTime']:
        total_cost += float(day['Total']['UnblendedCost']['Amount'])
    if total_cost > 50:
        alert = "High AWS cost detected"
    else:
        alert = "AWS cost is under control"
    message = f"{alert}\nWeekly AWS Cost: ${total_cost:.2f}"
```

```
sns.publish(TopicArn=TOPIC_ARN, Message=message, Subject="AWS Cost Alert")

file_name = f"report-{today}.txt"
s3.put_object(Bucket=BUCKET_NAME, Key=file_name, Body=message)

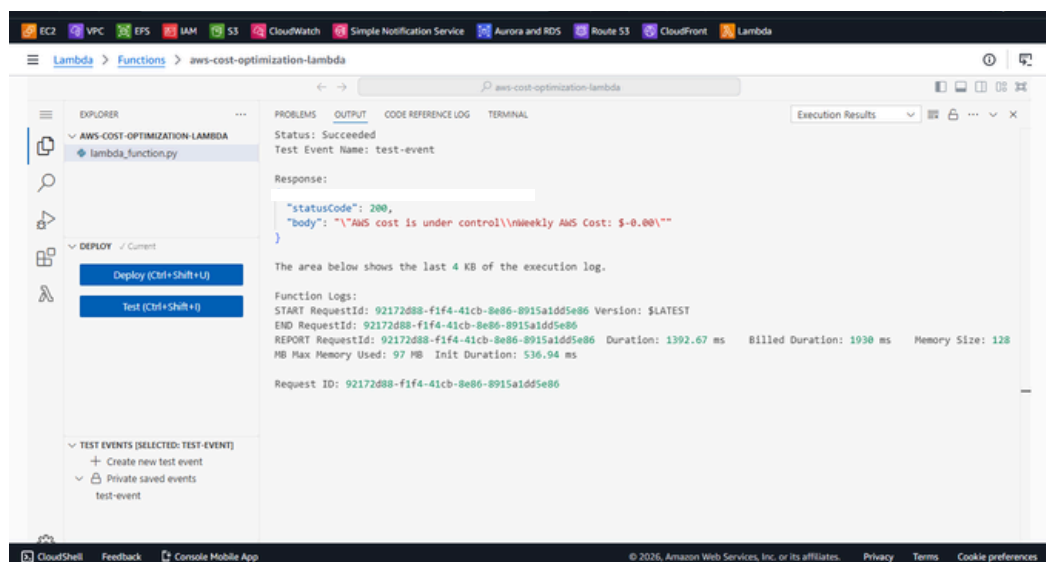
return {"statusCode": 200, "body": json.dumps(message)}
```

Note: The Lambda fetches a rolling 7-day cost window each time it runs. AWS Cost Explorer data has a delay of 8–24 hours, so reports reflect near real-time billing data, not instant figures.



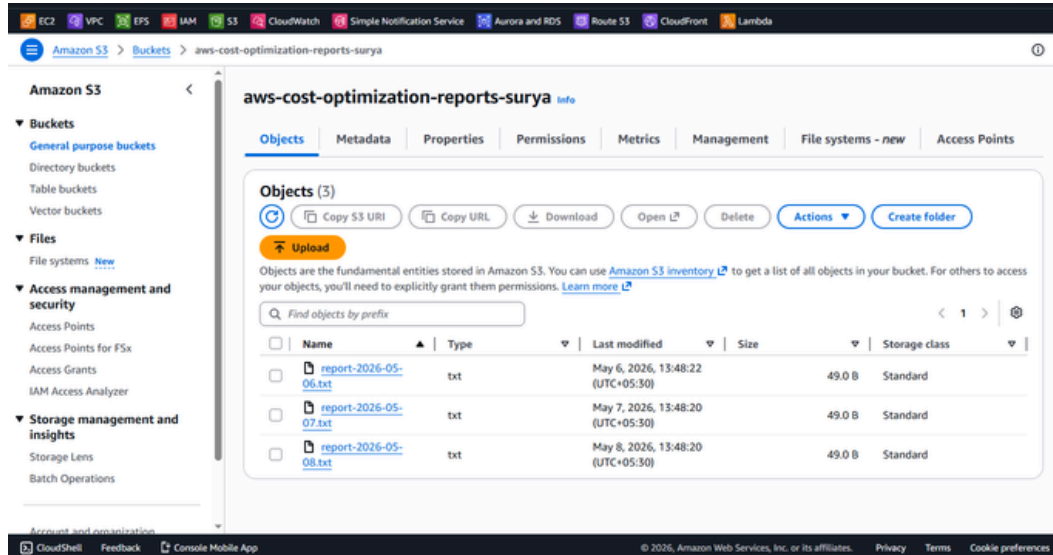
Step 11: Test the Lambda Function

1. Inside the Lambda page, click **Test** → **Create Test Event** (event name: test-event, keep the default JSON body). Click Test again to execute.
2. Lambda will run the full flow - call the billing API, send an email alert via SNS, and save a report file in S3. Check the Execution Results panel at the bottom for a success response.



Verify both outcomes after the test:

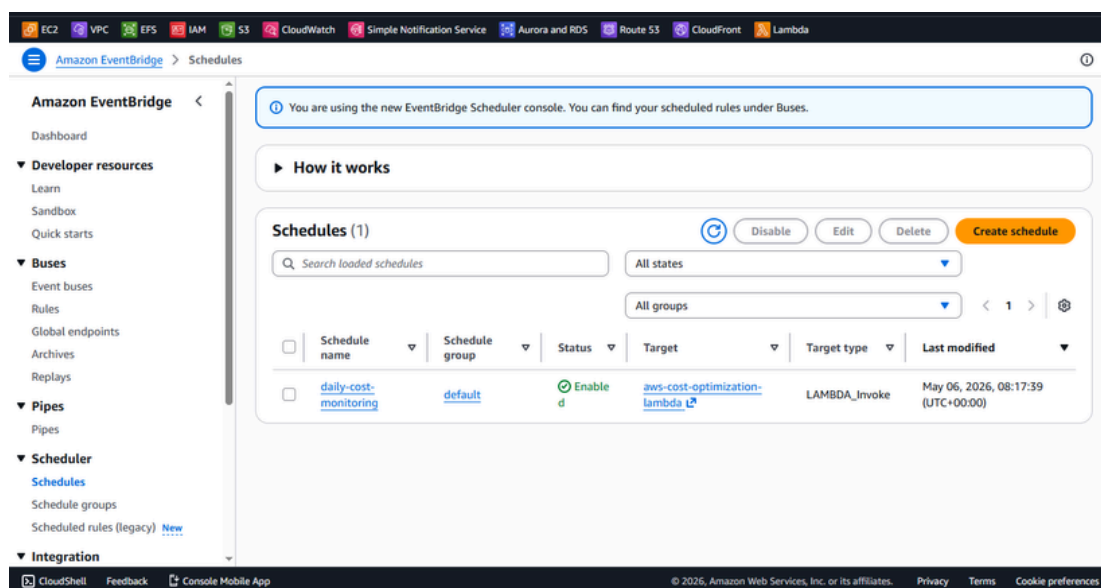
- Open your email inbox - you should receive an AWS Cost Alert email
- Open AWS Console → S3 → your bucket - you should see a report-YYYY-MM-DD.txt file



PHASE 3 – Automation and Scheduling

Step 12: Schedule Lambda with EventBridge (Daily Automation)

1. Instead of triggering Lambda manually every day, use AWS EventBridge Scheduler to run it automatically on a fixed daily schedule
2. Go to **AWS Console → EventBridge → Schedules → Create Schedule** and configure it as shown below. This ensures cost monitoring and reporting happens every day without any manual effort.



Field	Value
Schedule name	daily-cost-monitoring
Schedule pattern	Recurring → Rate-based
Rate	1 Day
Flexible time window	Off
Target type	AWS Lambda
Lambda function	aws-cost-optimization-lambda

PHASE 4 – Streamlit Analytics Dashboard

Step 13: Upgrade the Dashboard with Live AWS Data

1. Replace the basic dashboard code with the full version that connects to your S3 bucket, reads all cost report files, and displays live billing analytics
2. Open **dashboard.py** in VS Code, delete all existing content, and paste the upgraded code. This version shows total cost, average cost, latest weekly cost, a Plotly line chart for cost trends, and automated cost status insights based on threshold comparison.

```
import streamlit as st
import boto3, pandas as pd, re, plotly.express as px
from datetime import datetime

s3 = boto3.client('s3')
BUCKET_NAME = "aws-cost-optimization-reports-surya"

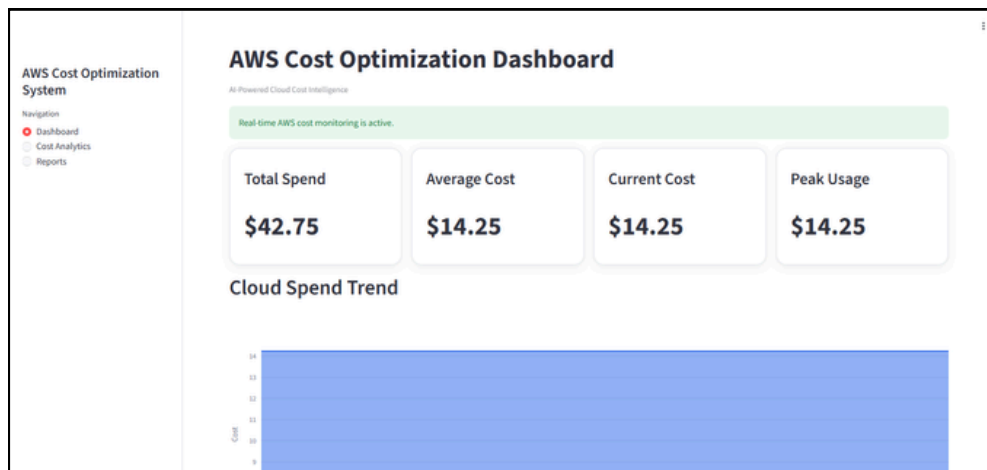
st.set_page_config(page_title="AWS Cost Optimization Dashboard",
page_icon="■", layout="wide")

# Fetch all report files from S3, parse cost values,
# build DataFrame, display metric cards and Plotly trend chart.
# Full source code available in dashboard.py
```

Save the file with Ctrl+S and run it locally to verify the dashboard loads S3 data correctly:

```
streamlit run dashboard.py
```

Streamlit will start a local web server and automatically open the dashboard in your browser at the address <http://localhost:8501>.

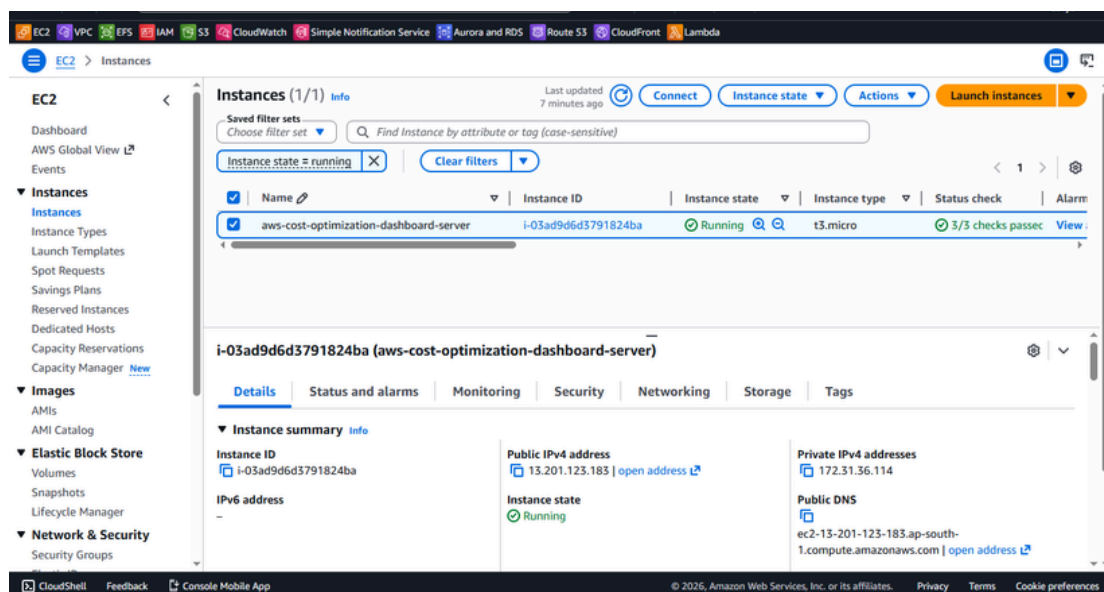


PHASE 5 — EC2 Deployment (Live Cloud Server)

Step 14: Launch an EC2 Instance on AWS

1. To host the dashboard live on the internet 24/7, launch an EC2 virtual machine on AWS.
2. Go to **AWS Console** → **EC2** → **Launch Instance** and configure it as shown. EC2 gives you a real Linux server in the cloud where the Streamlit app runs publicly and is accessible from any browser.

Field	Value
Name	aws-cost-optimization-dashboard-server
AMI	Ubuntu Server 24.04 LTS
Instance Type	t2.micro (Free Tier)
Key Pair	Create new → cloud-ai-key (.pem format)
Security Group Ports	SSH (22), HTTP (80), HTTPS (443), Custom TCP (8501)



Note: The .pem key file downloads automatically when you create the key pair. Store it safely inside your project folder - it cannot be downloaded again. Port 8501 is required for Streamlit to be publicly accessible.

Step 15: Connect to EC2 via SSH from Local Terminal

1. Use the downloaded .pem key to securely connect to your EC2 server from your local Ubuntu terminal. SSH (Secure Shell) creates an encrypted connection between your machine and the cloud server. The key file must have restricted permissions before SSH will accept it — this is a Linux security requirement

```
# Fix key file permissions (required by SSH)
chmod 400 cloud-ai-key.pem

# Connect to EC2 server
ssh -i cloud-ai-key.pem ubuntu@YOUR_PUBLIC_IP

# Type 'yes' when asked about fingerprint
```

Note: If you are on Windows using WSL (Windows Subsystem for Linux), first type 'wsl' to switch to the Linux environment, then run chmod and ssh commands from there.

Step 16: Install Required Packages on EC2

1. Once connected to EC2, update the server's package list and install all software needed to run the project. Ubuntu 24.04 uses a protected Python environment, so the **--break-system-packages** flag is required when using pip3 to install packages.

```
sudo apt update && sudo apt upgrade -y
sudo apt install python3-pip git -y

pip3 install streamlit boto3 pandas plotly --break-system-packages
```

Step 17: Clone GitHub Repo and Configure AWS CLI on EC2

1. Clone your project from GitHub to bring all the code onto the EC2 server. Then install AWS CLI and configure it with your credentials so the dashboard running on EC2 can connect to S3 and read the cost report files stored there.

```
# Clone the repo
git clone https://github.com/Username/repo.git

cd aws-cost-optimization-system/
```

```
# Install AWS CLI on EC2
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o
"awscliv2.zip"
sudo apt install unzip -y && unzip awscliv2.zip
sudo ./aws/install

# Configure AWS credentials
aws configure
# Enter: Access Key, Secret Key, ap-south-1, json

# Verify connection
aws sts get-caller-identity
```

Step 18: Run Streamlit Dashboard on EC2 and Verify Live Access

1. Start the Streamlit app on EC2 with the server address set to 0.0.0.0 so it is publicly accessible from any browser using the EC2 public IP. Open your browser and navigate to the URL below to confirm the dashboard is live on the internet.

```
~/local/bin/streamlit run dashboard.py \
--server.port 8501 --server.address 0.0.0.0
```

Open in browser to verify:

```
http://YOUR_EC2_PUBLIC_IP:8501
```

Step 19: Keep Dashboard Running 24/7 with PM2

1. Currently the dashboard stops as soon as you close the SSH terminal.
2. PM2 is a process manager for Node.js that can also manage any shell command as a background process - keeping the dashboard running permanently even after disconnecting from SSH.
3. Install Node.js and PM2, then register the Streamlit app as a managed background process.

```
sudo apt install nodejs npm -y
sudo npm install -g pm2

cd ~/aws-cost-optimization-system

# Start dashboard as background process using safer PM2 syntax
pm2 start streamlit --name aws-dashboard -- run dashboard.py \
--server.port 8501 --server.address 0.0.0.0

pm2 list
pm2 save
```

The dashboard is now permanently live. Visit the public IP at any time - even after closing your terminal or restarting your laptop - and the dashboard will still be accessible.

```
Use --update-env to update environment variables
[PM2] Applying action restartProcessId on app [aws-dashboard](ids: [ 0 ])
[PM2] [aws-dashboard](0) ✓
```

id	name	namespace	version	mode	pid	uptime	0	status	cpu	mem	user	watching
0	aws-dashboard	default	N/A	fork	28679	0s	1212	online	0%	4.7mb	ubuntu	disabled

Security Note - Port 8501 & Production Hardening

In this project, port 8501 is opened directly to the public internet (0.0.0.0/0) for simplicity and ease of access during development and demonstration. This is acceptable for a personal demo or portfolio project, but is not recommended for a production environment.

In a proper production deployment, the recommended approach is:

- Close port 8501 from public access in the security group
- Install Nginx as a reverse proxy to forward traffic from port 80/443 to port 8501 internally
- Attach an SSL certificate (via Let's Encrypt / AWS Certificate Manager) to enable HTTPS
- Use a custom domain name instead of a raw IP address

This ensures the dashboard is served securely over HTTPS on standard ports, which is the industry-standard approach for hosting any web application on a cloud server.

PHASE 6 – Automated Deployment Pipeline with GitHub Actions

Step 20: Create the GitHub Actions Workflow File

1. You already created the file `.github/workflows/deploy.yml` back in Step 1 as part of the initial project structure. Now open it in VS Code and add the actual pipeline code
2. GitHub Actions is a CI/CD automation tool built directly into GitHub - every time you push new code to the main branch, it automatically SSHs into your EC2 server, pulls the latest code, and restarts the Streamlit process via PM2.
3. This eliminates the need to manually connect to EC2 every time you make a change to your project.
4. Open `.github/workflows/deploy.yml`. Paste the configuration and save the file with **Ctrl+S**.

```
name: Deploy AWS Cost Optimization System

on:
  push:
    branches:
      - main
```

```
jobs:
  deploy:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout Repository
        uses: actions/checkout@v4

      - name: Deploy to EC2
        uses: appleboy/ssh-action@master
        with:
          host: ${ secrets.EC2_HOST }
          username: ${ secrets.EC2_USER }
          key: ${ secrets.EC2_SSH_KEY }
          script: |
            cd ~/aws-cost-optimization-system
            git pull origin main
            pm2 restart aws-dashboard
```

Step 21: Add GitHub Secrets for EC2 Access

1. The workflow references **secrets.EC2_HOST**, **secrets.EC2_USER** and **secrets.EC2_SSH_KEY** - these are secure environment variables stored inside GitHub so your EC2 IP address and private key are never exposed directly in the code.
2. Go to your **GitHub repository** → **Settings** → **Secrets and Variables** → **Actions** → **New Repository Secret** and add both secrets one by one.

Secrets Name	Value
EC2_HOST	Your EC2 public IP address (e.g. 13.201.123.183)
EC2_USER	Your username (e.g. ubuntu)
EC2_SSH_KEY	Full contents of your cloud-ai-key.pem file

Repository secrets			New repository secret	
Name		Last updated		
EC2_HOST		42 minutes ago		
EC2_SSH_KEY		29 minutes ago		
EC2_USER		42 minutes ago		

3. To get the full contents of your .pem key, run this in your local terminal:

```
cat cloud-ai-key.pem
```

4. Copy the entire output including the -----BEGIN RSA PRIVATE KEY----- and -----END RSA PRIVATE KEY----- lines and paste it as the value for EC2_SSH_KEY.

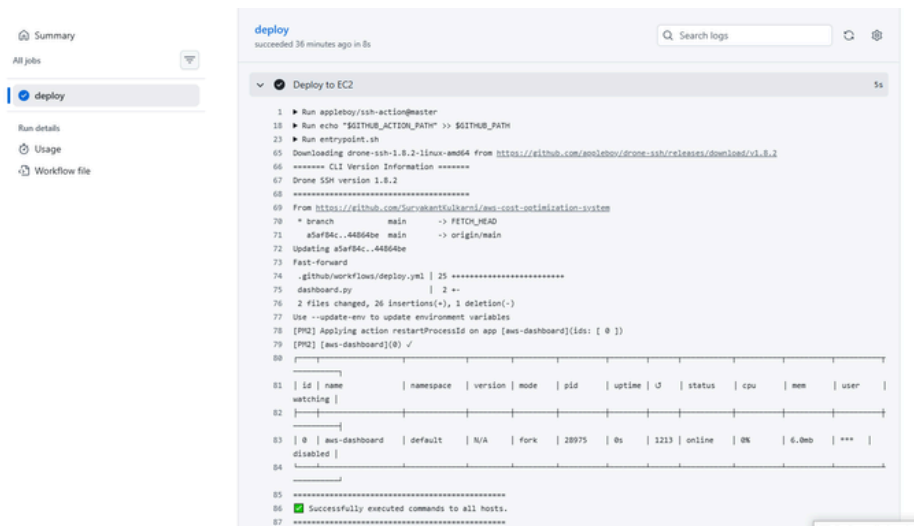
Step 22: Push Code to Trigger the Pipeline

1. Test the automated deployment pipeline by making a small change to any file - for example, add a comment line in **dashboard.py** - and pushing it to the main branch. This push will automatically trigger the GitHub Actions workflow.
2. Go to your **GitHub repository** → **Actions** tab to watch the pipeline run in real time with each step visible as it executes.

```
git add .
git commit -m "Added deployment pipeline via GitHub Actions"
git push origin main
```

Step 23: Verify Successful Deployment

1. Once the workflow finishes, every step inside the pipeline will show a green checkmark. This confirms that GitHub Actions successfully connected to your EC2 server via SSH, pulled the latest code from the repository, and restarted the Streamlit dashboard using PM2 - all automatically without any manual steps from your side.

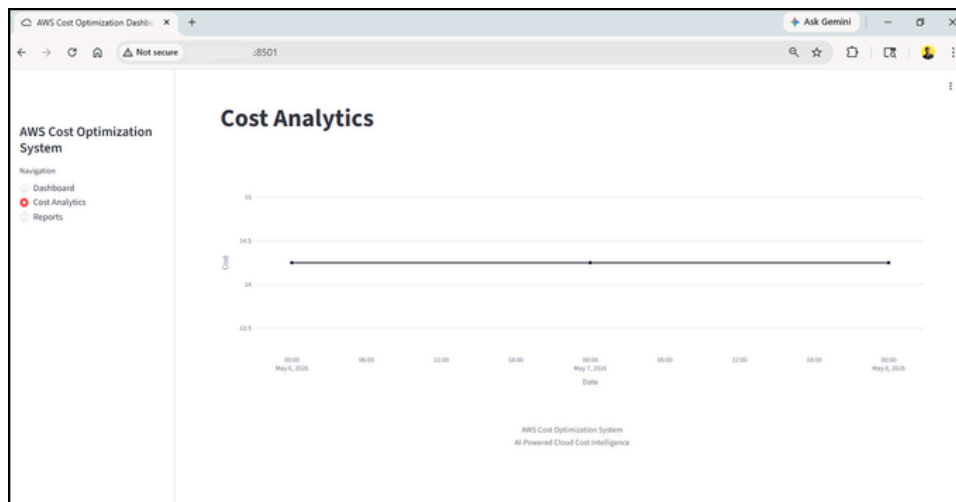
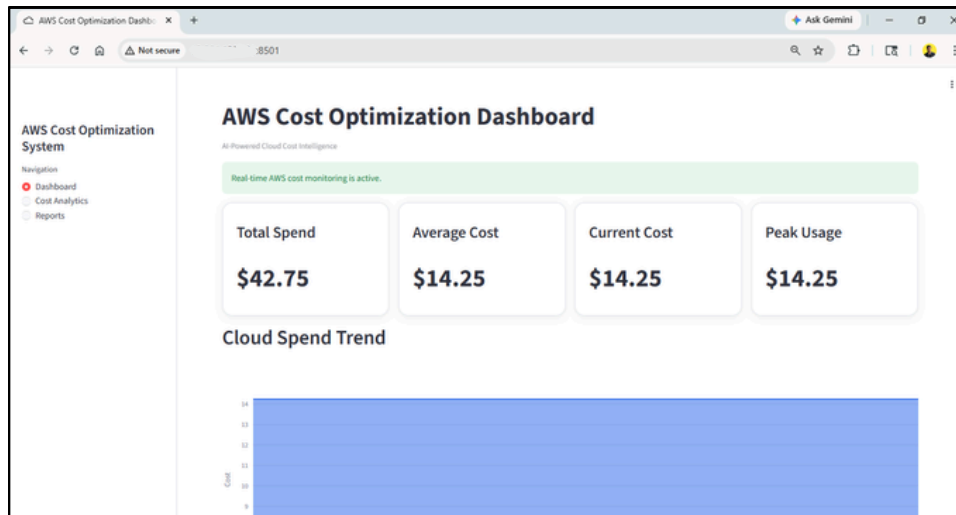


Note: This pipeline implements Continuous Delivery (CD) - automated deployment triggered on every git push to main. It is not a full CI/CD pipeline as there are no testing, linting, or build validation stages. To make it a complete CI/CD pipeline in the future, you can add steps before the deploy job for running pytest, flake8 linting, or code quality checks.

2. Open your browser and visit

```
http://YOUR_EC2_PUBLIC_IP:8501
```

Final screenshot of the live dashboard at http://PUBLIC_IP:8501 fully working with charts and data



The screenshot displays the 'Historical Reports' section of the dashboard. It contains a table with the following data:

Date	Cost
0 2025-05-06 00:00:00	14.25
1 2025-05-07 00:00:00	14.25
2 2025-05-08 00:00:00	14.25

The navigation sidebar on the left has 'Reports' selected.

TECHNOLOGIES USED

Category	Technology	Purpose
Frontend	Streamlit + Plotly	Interactive web dashboard and charts
Backend	Python 3 + Boto3	Application logic and AWS SDK integration
Serverless	AWS Lambda (Python 3.12)	Automated billing data collection
Scheduling	Amazon EventBridge	Trigger Lambda function daily
Storage	Amazon S3	Store cost reports as text files
Alerts	Amazon SNS	Send email notifications
Billing API	AWS Cost Explorer	Fetch AWS spending data
Hosting	AWS EC2 (Ubuntu 24.04)	Run the Streamlit dashboard publicly
Security	AWS IAM	Manage permissions for all AWS services
Process Mgmt	PM2 (Node.js)	Keep dashboard running in background
Version Control	Git + GitHub	Code management and deployment
CI/CD	GitHub Actions	Automatic deployment on code push

PROJECT OUTCOME

This project successfully delivers a complete cloud-native AWS Cost Optimization System that is fully automated, publicly accessible, and deployment-ready. Starting from a blank folder on a local machine, the project progresses through local development, AWS backend configuration, EC2 server deployment, process management, and finally a complete CI/CD pipeline - covering the full DevOps lifecycle from development to production.

The deployed system collects billing data every day without manual intervention, stores historical cost reports in Amazon S3, sends email alerts when spending is too high, and presents all of that data through a professional interactive dashboard that anyone can access from any browser. This project demonstrates strong practical skills in AWS, Python, Linux, DevOps tooling, and cloud architecture that are directly relevant to real-world cloud and DevOps engineering roles.